DANA PREDICTOR

MAPA DE RIESGO DE INUNDACIÓN URBANA

FASE 2: PRODUCTO FUNCIONAL LOCAL Y REPOSITORIO REPRODUCIBLE



GRUPO 10 PROYECTO FINAL DE BIG DATA E INTELIGENCIA ARTIFICIAL

DENNIS BONILLA

RAFAEL CHAMBILLA

VÍCTOR MARTÍNEZ

JAUME PALAU

12/11/2026

Tabla de contenidos

1. Portada 3

2. Estado actual del producto 3

3. Guía de ejecución local 4

4. Evidencias funcionales 5

5. Arquitectura implementada 9

6. Índice de código 10

7. Repositorio versionado y reproducible 11

8. Explicación por módulo 11

9. Limitaciones actuales 13

10. Lista de comprobación de reproducibilidad 14

1. Portada

- **Proyecto:** DANA Predictor, mapa de susceptibilidad a inundación aprendido de la DANA de 2024 en València, L'Horta Sud y L'Horta Oest
- **Integrantes (Grupo 10):** Dennis Bonilla, Rafael Chambilla, Víctor Martínez, Jaume Palau
- **Fecha:** 12/11/2026
- **Autor del documento:** Grupo 10
- **Repositorio:** https://github.com/Victorproyecto/IA_Project
- **Versión entregada:** tag `nf3-entrega` de la rama `main`

2. Estado actual del producto

Completo

- Ingesta reproducible de 11 fuentes reales desde sus descargas originales (`src/pipeline/ingest.py`), incluida la reparación de la huella oficial de la GVA, que llega con geometría inválida, y el cálculo de la altura sobre el cauce (HAND) siguiendo el flujo del agua sobre el MDT. Los datos no se versionan: los originales (`data/raw`, 1,7 GB) están en Google Drive y el resto se genera con `ingest`, `run` y `train`.
- Preparación: 41.401 celdas de 100 m en los 22 municipios del área (València, L'Horta Sud y L'Horta Oest), 9 variables (altitud, pendiente, suelo sellado, distancia al cauce principal, altura sobre el cauce y 4 de geología) y etiqueta de la DANA (Copernicus EMS + huella de la GVA: 9.765 celdas inundadas, el 24 %). Se excluyen el mar, los arrozales y el agua permanente.
- Entrenamiento de 2 modelos con validación cruzada por bloques espaciales de 1 km, comparación con PATRICOVA y el SNCZI, prueba externa en la Ribera (otra cuenca), comprobación con calados de campo, ficha del modelo y predicciones.
- Aplicación: mapa de celdas de 100 m con JavaScript (MapLibre) y, a su lado, la ficha de la celda pulsada (nivel, posición en el área, terreno, mapas oficiales y lo observado en la DANA); buscador con sugerencias de municipios, barrios, calles y direcciones; capas opcionales del límite del área y de lo inundado en la DANA; tabla accesible con descarga en CSV; ficha del modelo; tema claro y oscuro y números en formato español.
- 34 tests automáticos (también en Windows), Docker y CI.

Parcial

- El modelo (Random Forest `0.4.0-rf`) cumple los dos criterios de la Fase 1: Recall 0,80 y **F1 0,722 frente al objetivo de 0,70, con poco margen**. El F1 hay que leerlo frente a 0,38, que es lo que daría marcar todas las celdas como inundadas; el ROC-AUC es 0,92. El objetivo se alcanza al añadir la altura sobre el cauce (sin ella, 0,687) y con el método de validación del proyecto (bloques de 1 km): con bloques de 3 km el F1 es 0,63 con y sin esa variable (detalle en `docs/MEJORAS_MODELO_NF3.md`, apartado 2.5).

- Dentro del área supera a los mapas oficiales: F1 0,72 frente a 0,65 del SNCZI (T=500) y 0,46 de PATRICOVA. La comparación favorece algo al modelo, que aprende de esta misma DANA, mientras que los mapas oficiales son anteriores. Aun así, el 62 % de las celdas inundadas (6.102 de 9.765) estaba fuera de PATRICOVA.
- **En otra cuenca no generaliza:** en la Ribera (que el modelo nunca ve) su ROC-AUC es 0,59, mientras que PATRICOVA y el SNCZI alcanzan 0,80 y 0,78. Con un solo episodio, el modelo aprende cómo se inundó cada zona en 2024, no reglas que se trasladen a otra.
- En la CI se saltan los tests que necesitan datos, porque `data/` no se versiona. En local, con los datos, pasan los 34.

Cambio de alcance respecto a la Fase 1

La Fase 1 limitaba el piloto a la ciudad de València. Se amplía a L'Horta Sud y L'Horta Oest porque es donde la DANA causó más daño y donde el producto tiene más sentido, y porque permite medir si el modelo generaliza a una cuenca que no ha visto (la Ribera). Detalle en `docs/MEJORAS_MODELO_NF3.md`.

3. Guía de ejecución local

Requisitos: Python 3.12 (o Docker), unos 3 GB libres y los datos originales (1,7 GB). Están en un zip en Google Drive (https://drive.google.com/file/d/176-uFyjY6HrutmeO81buOrQoFvy12swU/view) que se descomprime dentro de `data/`, de modo que quede `data/raw/` (detalle en `docs/DATOS.md`). El navegador necesita conexión a internet para el mapa de fondo, la librería del mapa y las sugerencias de direcciones.

Variables de entorno: ninguna obligatoria. `DATA_DIR` permite usar otra carpeta de datos.

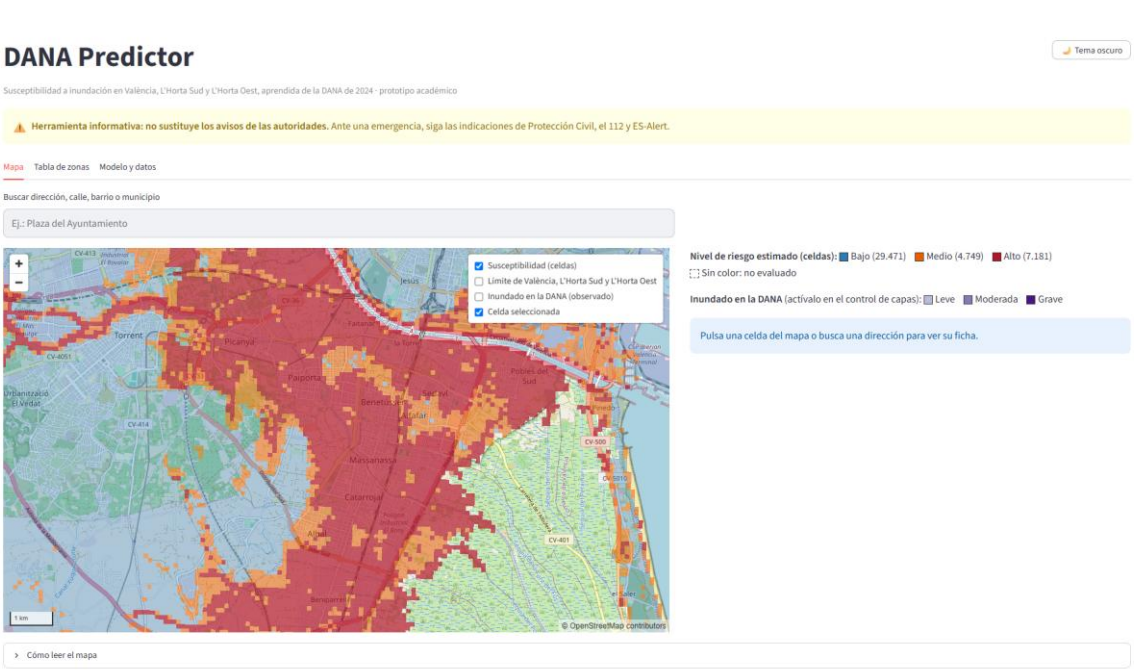
```
git clone https://github.com/Victorproyecto/IA_Project.git
cd IA_Project
git checkout nf3-entrega
python3 -m venv .venv && source .venv/bin/activate
pip install -r environment/requirements.txt
# antes: descomprimir el zip de Google Drive dentro de data/ (queda data/raw/)
python -m src.pipeline.ingest
python -m src.pipeline.run
python -m src.pipeline.train
streamlit run src/frontend/app.py      # http://localhost:8501
python -m pytest
```

Orden de arranque: ingest → run → train → app. Cada paso lee lo que escribe el anterior. Con Docker, los mismos pasos están en el README.

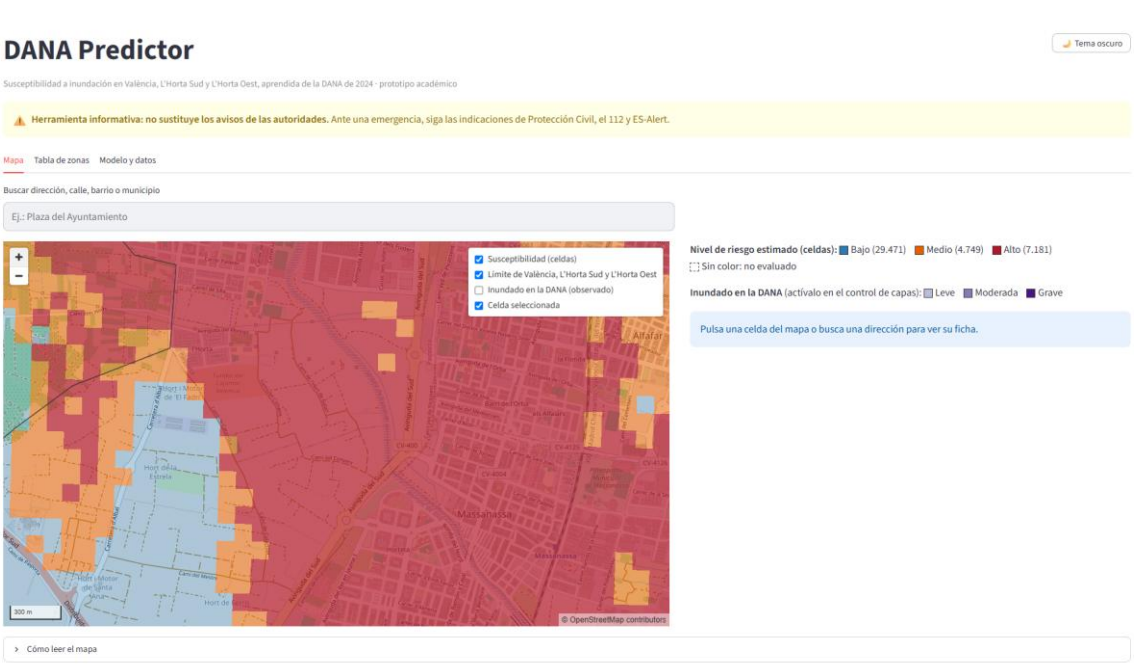
4. Evidencias funcionales

Capturas de la aplicación en local (http://localhost:8501) y de la terminal, con el código de esta versión. Las de terminal muestran la salida real de cada comando, tomada de su registro, sin los avisos de las librerías. Capturas a 1920 × 1080 en tema claro.

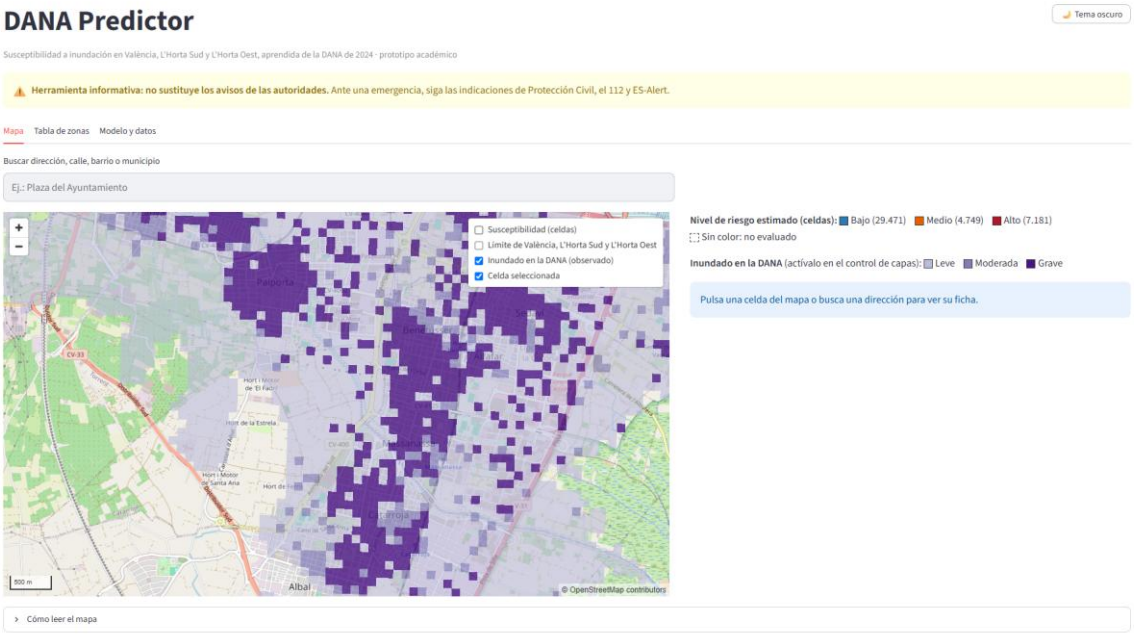
Front funcionando: mapa de susceptibilidad. Cada color es una celda de 100 m, en València, L'Horta Sud y L'Horta Oest. A la derecha, la leyenda con el número de celdas de cada nivel.



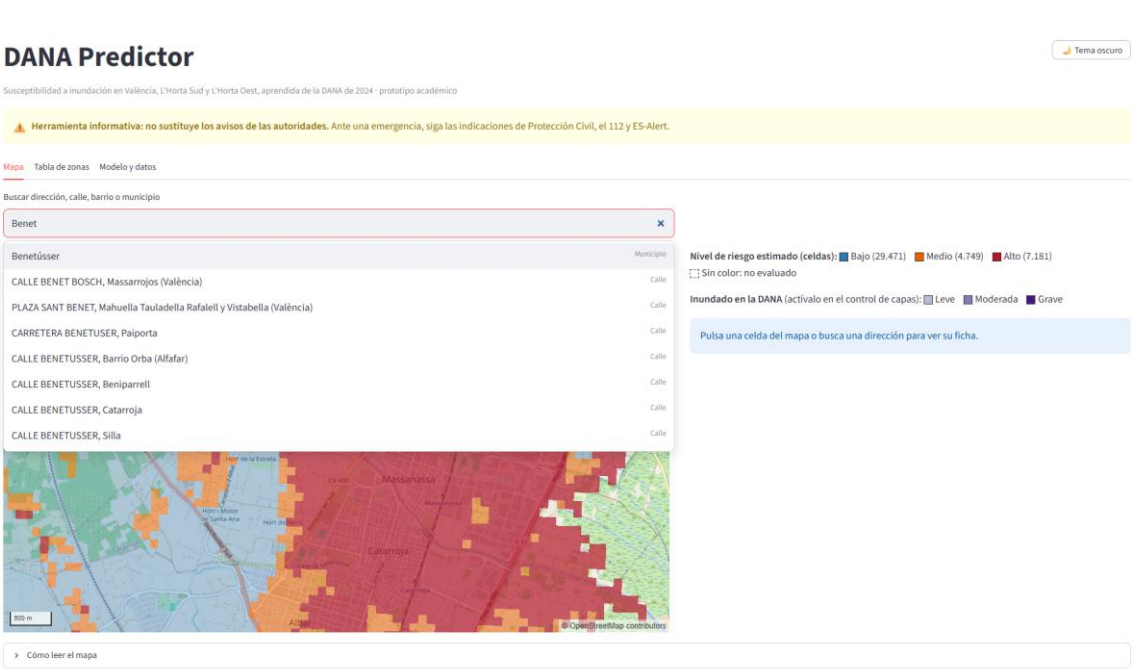
Front: detalle de la zona cero. Massanassa y Catarroja al acercar el mapa: cada cuadrado es una celda de 100 m. La línea gris es el límite entre València y L'Horta Sud (capa opcional).



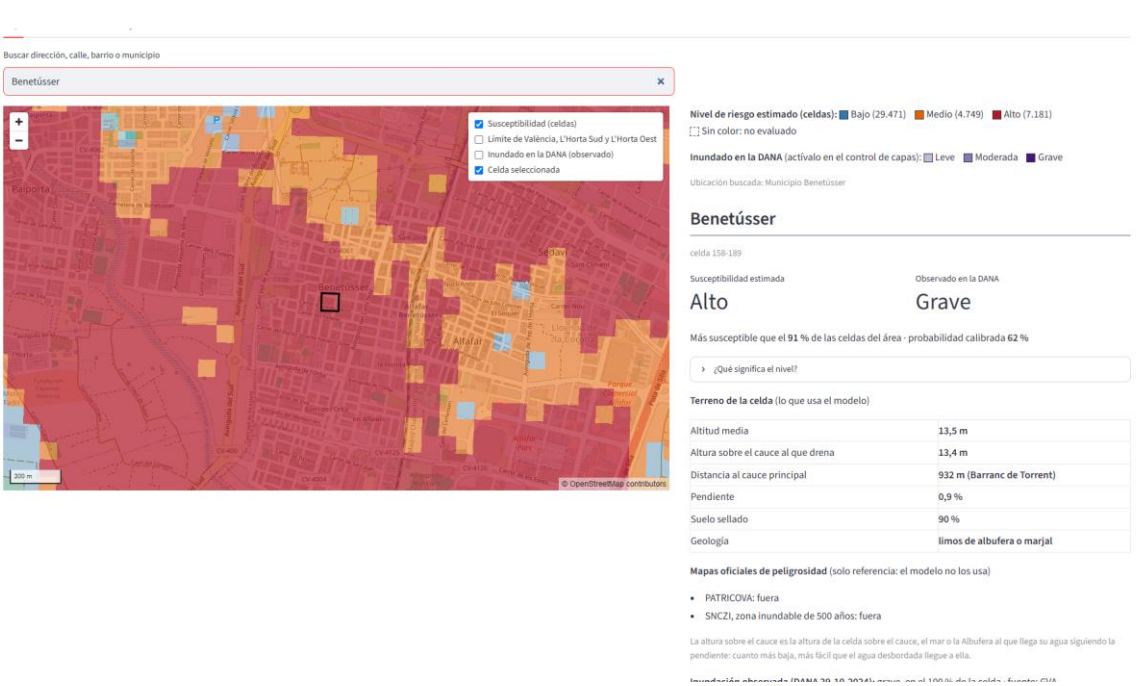
Front: capa de lo inundado en la DANA (Copernicus EMS y huella oficial de la GVA), en morado según la gravedad, para compararla con el mapa de susceptibilidad.



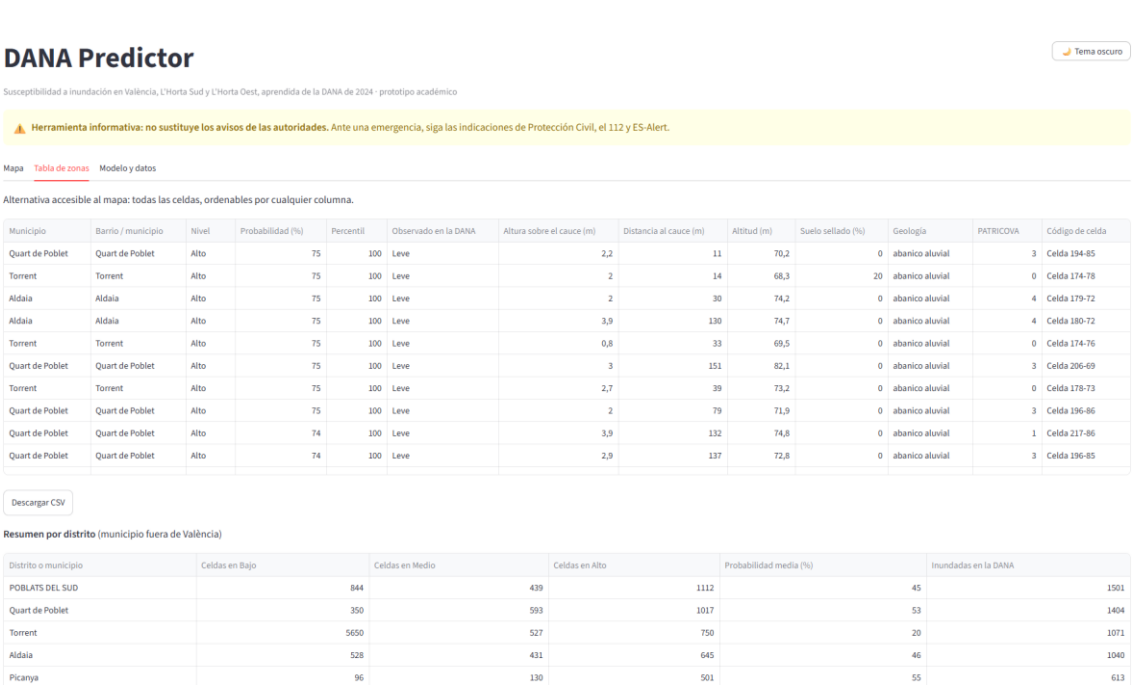
Front: buscador con sugerencias. Al escribir, sugiere al instante municipios, distritos y barrios y, a partir de 3 letras, calles y direcciones del área (CartoCiudad, del IGN).



Backend activo: búsqueda de un municipio y ficha de la celda. El frontend pide a `services.py` la celda, su nivel, la posición relativa, el terreno, los mapas oficiales y lo observado con su fuente, y la ficha aparece junto al mapa. Las funciones del backend también las comprueba `tests/test_backend.py` (captura de pytest en el apartado 7).



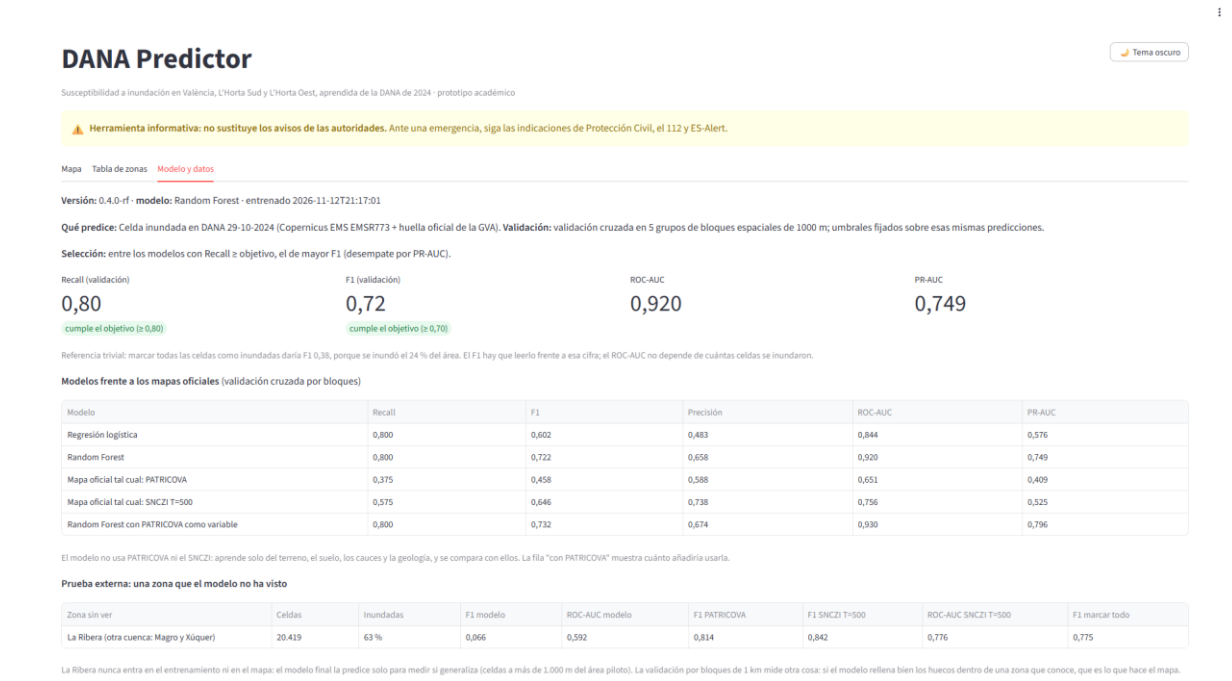
Front: tabla accesible y resumen por distrito o municipio



Municipio	Barrio / municipio	Nivel	Probabilidad (%)	Percentil	Observado en la DANA	Altura sobre el cauce (m)	Distancia al cauce (m)	Altitud (m)	Suelo sellado (%)	Geología	PATRICOVA	Código de celda
Quart de Poblet	Quart de Poblet	Alto	75	100	Leve	2,2	11	70,2	0	abanico aluvial	3	Celda 194-85
Torrent	Torrent	Alto	75	100	Leve	2	14	68,3	20	abanico aluvial	0	Celda 174-78
Aldaia	Aldaia	Alto	75	100	Leve	2	30	74,2	0	abanico aluvial	4	Celda 179-72
Aldaia	Aldaia	Alto	75	100	Leve	3,9	130	74,7	0	abanico aluvial	4	Celda 180-72
Torrent	Torrent	Alto	75	100	Leve	0,8	33	69,5	0	abanico aluvial	0	Celda 174-76
Quart de Poblet	Quart de Poblet	Alto	75	100	Leve	3	151	82,1	0	abanico aluvial	3	Celda 206-69
Torrent	Torrent	Alto	75	100	Leve	2,7	39	73,2	0	abanico aluvial	0	Celda 178-73
Quart de Poblet	Quart de Poblet	Alto	75	100	Leve	2	79	71,9	0	abanico aluvial	3	Celda 196-86
Quart de Poblet	Quart de Poblet	Alto	74	100	Leve	3,9	132	74,8	0	abanico aluvial	1	Celda 217-86
Quart de Poblet	Quart de Poblet	Alto	74	100	Leve	2,9	137	72,8	0	abanico aluvial	3	Celda 196-85

Distrito o municipio	Celdas en Bajo	Celdas en Medio	Celdas en Alto	Probabilidad media (%)	Inundadas en la DANA
POBLATS DEL SUD	844	439	1112	45	1501
Quart de Poblet	350	593	1017	53	1404
Torrent	5650	527	750	20	1071
Aldaia	528	431	645	46	1040
Picanya	96	130	501	55	613

Flujo completo: ficha del modelo en la aplicación. Comparativa con PATRICOVA y el SNCZI, prueba externa en la Ribera, referencia trivial y comprobación con calados de campo.



DANA Predictor

Tema oscuro

Susceptibilidad a inundación en València, L'Horta Sud y L'Horta Oest, aprendida de la DANA de 2024 · prototipo académico

⚠ **Herramienta informativa: no sustituye los avisos de las autoridades.** Ante una emergencia, siga las indicaciones de Protección Civil, el 112 y ES-Alert.

Mapa | Tabla de zonas | Modelo y datos

Versión: 0.4.0-rf · **modelo:** Random Forest · entrenado 2026-11-12T21:17:01

Qué predice: Celda inundada en DANA 29-10-2024 (Copernicus EMS EMSR773 + huella oficial de la GVA). **Validación:** validación cruzada en 5 grupos de bloques espaciales de 1000 m; umbrales fijados sobre esas mismas predicciones.

Selección: entre los modelos con Recall ≥ objetivo, el de mayor F1 (desempate por PR-AUC).

Recall (validación)	F1 (validación)	ROC-AUC	PR-AUC
0,80	0,72	0,920	0,749
cumple el objetivo (≥ 0,80)	cumple el objetivo (≥ 0,70)		

Referencia trivial: marcar todas las celdas como inundadas daría F1 0,38, porque se inundó el 24 % del área. El F1 hay que leerlo frente a esa cifra; el ROC-AUC no depende de cuántas celdas se inundaron.

Modelos frente a los mapas oficiales (validación cruzada por bloques)

Modelo	Recall	F1	Precisión	ROC-AUC	PR-AUC
Regresión logística	0,800	0,602	0,483	0,844	0,576
Random Forest	0,800	0,722	0,658	0,920	0,749
Mapa oficial tal cual: PATRICOVA	0,375	0,458	0,588	0,651	0,409
Mapa oficial tal cual: SNCZI T=500	0,575	0,646	0,738	0,756	0,525
Random Forest con PATRICOVA como variable	0,800	0,732	0,674	0,930	0,796

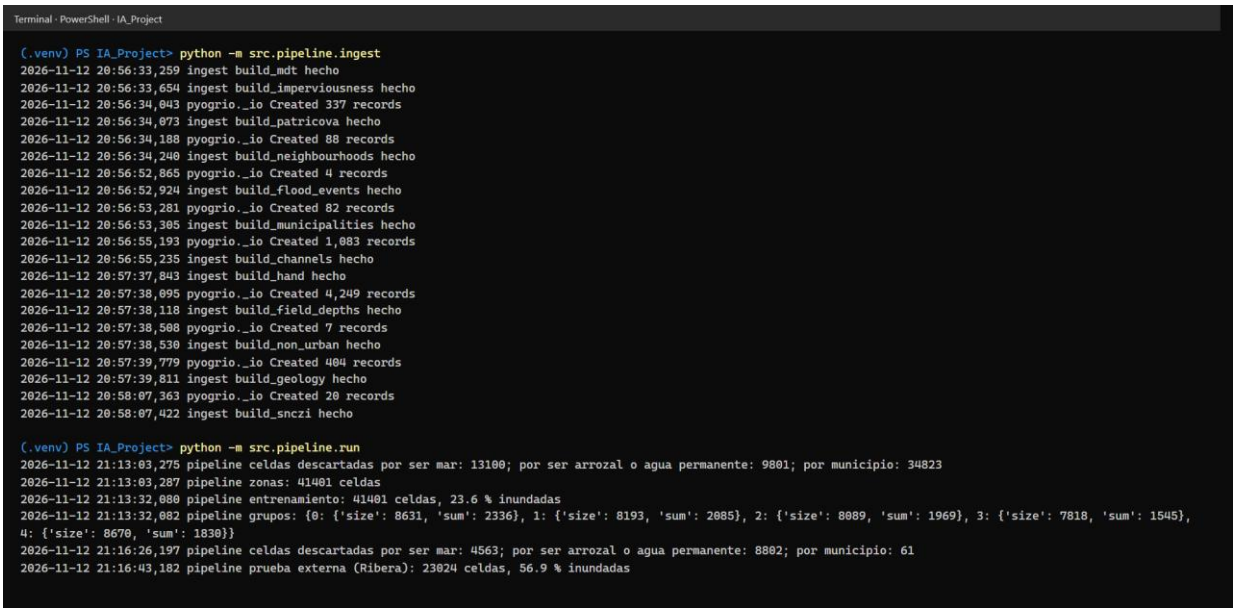
El modelo no usa PATRICOVA ni el SNCZI: aprende solo del terreno, el suelo, los cauces y la geología, y se compara con ellos. La fila "con PATRICOVA" muestra cuánto añadiría usarla.

Prueba externa: una zona que el modelo no ha visto

Zona sin ver	Celdas	Inundadas	F1 modelo	ROC-AUC modelo	F1 PATRICOVA	F1 SNCZI T=500	ROC-AUC SNCZI T=500	F1 marcar todo
La Ribera (otra cuenca: Magro y Xúquer)	20.419	63 %	0,066	0,592	0,814	0,842	0,776	0,775

La Ribera nunca entra en el entrenamiento ni en el mapa: el modelo final la predice solo para medir si generaliza (celdas a más de 1.000 m del área piloto). La validación por bloques de 1 km mide otra cosa: si el modelo rellena bien los huecos dentro de una zona que conoce, que es lo que hace el mapa.

Flujo completo en terminal: ingesta y preparación

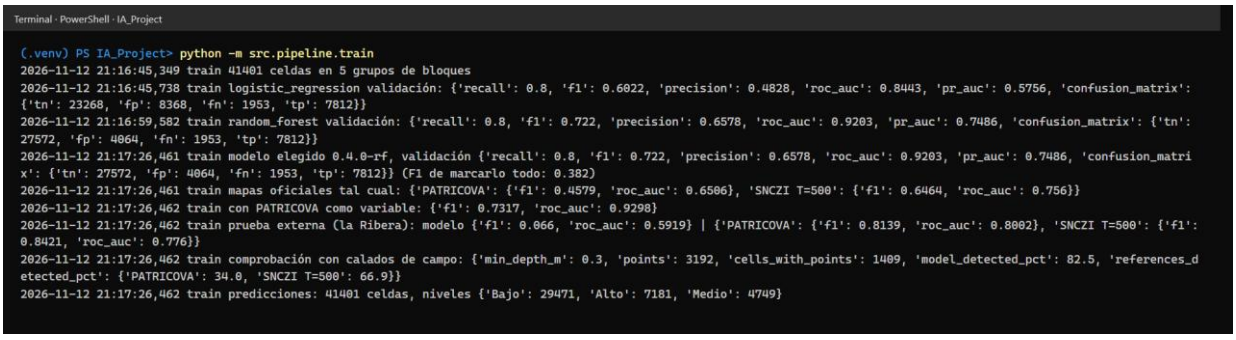


```
Terminal · PowerShell · IA_Project

(.venv) PS IA_Project> python -m src.pipeline.ingest
2026-11-12 20:56:33,259 ingest build_mdt hecho
2026-11-12 20:56:33,654 ingest build_imperviousness hecho
2026-11-12 20:56:34,043 pyogrio._io Created 337 records
2026-11-12 20:56:34,073 ingest build_patricova hecho
2026-11-12 20:56:34,188 pyogrio._io Created 88 records
2026-11-12 20:56:34,240 ingest build_neighbourhoods hecho
2026-11-12 20:56:52,865 pyogrio._io Created 4 records
2026-11-12 20:56:52,924 ingest build_flood_events hecho
2026-11-12 20:56:53,281 pyogrio._io Created 82 records
2026-11-12 20:56:53,305 ingest build_municipalities hecho
2026-11-12 20:56:55,193 pyogrio._io Created 1,083 records
2026-11-12 20:56:55,235 ingest build_channels hecho
2026-11-12 20:57:37,843 ingest build_hand hecho
2026-11-12 20:57:38,095 pyogrio._io Created 4,249 records
2026-11-12 20:57:38,118 ingest build_field_depths hecho
2026-11-12 20:57:38,508 pyogrio._io Created 7 records
2026-11-12 20:57:38,530 ingest build_non_urban hecho
2026-11-12 20:57:39,779 pyogrio._io Created 404 records
2026-11-12 20:57:39,811 ingest build_geology hecho
2026-11-12 20:58:07,363 pyogrio._io Created 20 records
2026-11-12 20:58:07,422 ingest build_snczi hecho

(.venv) PS IA_Project> python -m src.pipeline.run
2026-11-12 21:13:03,275 pipeline celdas descartadas por ser mar: 13100; por ser arrozal o agua permanente: 9801; por municipio: 34823
2026-11-12 21:13:03,287 pipeline zonas: 41401 celdas
2026-11-12 21:13:32,080 pipeline entrenamiento: 41401 celdas, 23.6 % inundadas
2026-11-12 21:13:32,082 pipeline grupos: {0: {'size': 8631, 'sum': 2336}, 1: {'size': 8193, 'sum': 2085}, 2: {'size': 8089, 'sum': 1969}, 3: {'size': 7818, 'sum': 1545},
4: {'size': 8670, 'sum': 1830}}
2026-11-12 21:16:26,197 pipeline celdas descartadas por ser mar: 4563; por ser arrozal o agua permanente: 8802; por municipio: 61
2026-11-12 21:16:43,182 pipeline prueba externa (Ribera): 23024 celdas, 56.9 % inundadas
```

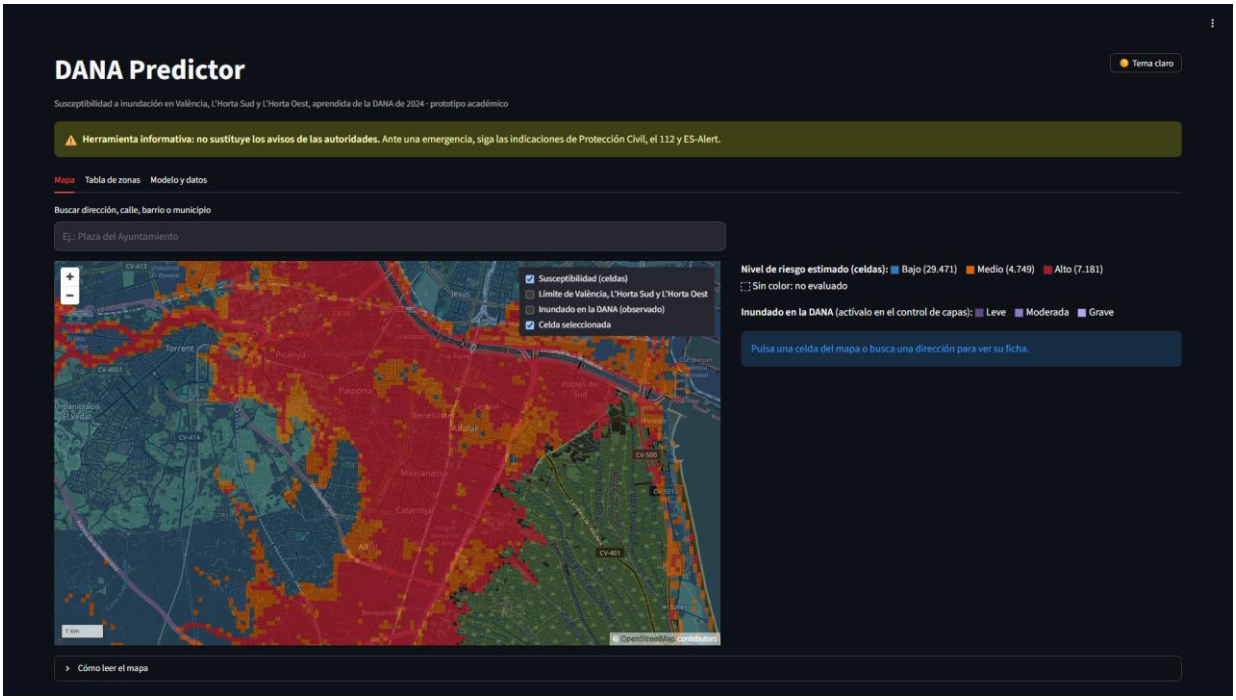
Flujo completo en terminal: entrenamiento



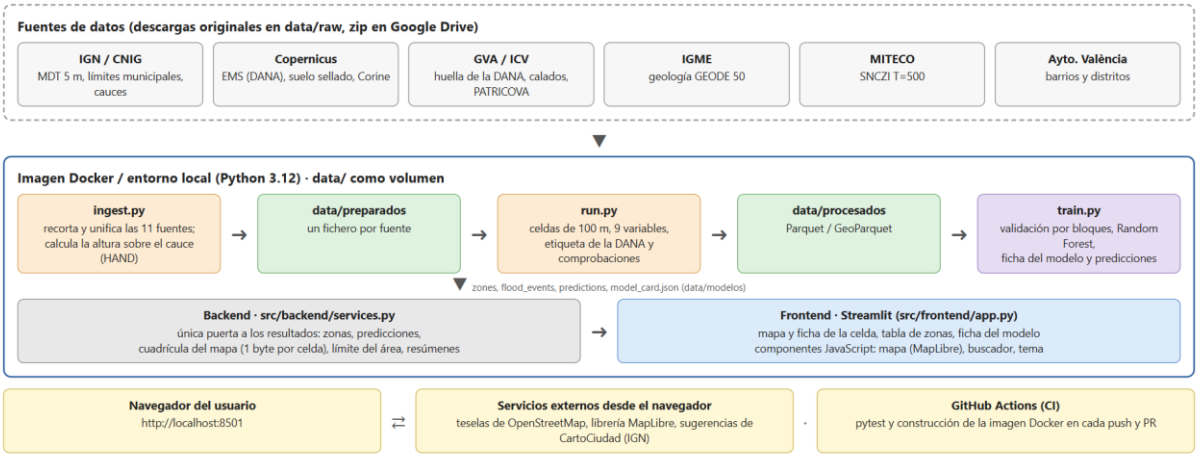
```
Terminal · PowerShell · IA_Project

(.venv) PS IA_Project> python -m src.pipeline.train
2026-11-12 21:16:45,349 train 41401 celdas en 5 grupos de bloques
2026-11-12 21:16:45,738 train logistic_regression validación: {'recall': 0.8, 'f1': 0.6022, 'precision': 0.4828, 'roc_auc': 0.8443, 'pr_auc': 0.5756, 'confusion_matrix':
{'tn': 23268, 'fp': 8368, 'fn': 1953, 'tp': 7812}}
2026-11-12 21:16:59,582 train random_forest validación: {'recall': 0.8, 'f1': 0.722, 'precision': 0.6578, 'roc_auc': 0.9203, 'pr_auc': 0.7486, 'confusion_matrix': {'tn':
27572, 'fp': 4064, 'fn': 1953, 'tp': 7812}}
2026-11-12 21:17:26,461 train modelo elegido 0.4.0-rf, validación {'recall': 0.8, 'f1': 0.722, 'precision': 0.6578, 'roc_auc': 0.9203, 'pr_auc': 0.7486, 'confusion_matri
x': {'tn': 27572, 'fp': 4064, 'fn': 1953, 'tp': 7812}} (F1 de marcarlo todo: 0.382)
2026-11-12 21:17:26,461 train mapas oficiales tal cual: {'PATRICOVA': {'f1': 0.4579, 'roc_auc': 0.6506}, 'SNCZI T=500': {'f1': 0.6464, 'roc_auc': 0.756}}
2026-11-12 21:17:26,462 train con PATRICOVA como variable: {'f1': 0.7317, 'roc_auc': 0.9298}
2026-11-12 21:17:26,462 train prueba externa (la Ribera): modelo {'f1': 0.066, 'roc_auc': 0.5919} | {'PATRICOVA': {'f1': 0.8139, 'roc_auc': 0.8802}, 'SNCZI T=500': {'f1':
0.8421, 'roc_auc': 0.776}}
2026-11-12 21:17:26,462 train comprobación con calados de campo: {'min_depth_m': 0.3, 'points': 3192, 'cells_with_points': 1409, 'model_detected_pct': 82.5, 'references_d
etected_pct': {'PATRICOVA': 34.0, 'SNCZI T=500': 66.9}}
2026-11-12 21:17:26,462 train predicciones: 41401 celdas, niveles {'Bajo': 29471, 'Alto': 7181, 'Medio': 4749}
```

Front: tema oscuro. El botón de la esquina superior derecha cambia entre tema claro y oscuro; el mapa de fondo se oscurece con el tema.



5. Arquitectura implementada



- **Pipeline por lotes:** se ejecuta una vez y deja los resultados en Parquet / GeoParquet. No hay base de datos.
- **Backend:** módulo Python que lee esos resultados. El frontend solo accede a los datos a través de él.
- **Frontend:** Streamlit, en el mismo proceso que el backend, con tres componentes JavaScript propios: el mapa de celdas (MapLibre GL JS), el buscador con sugerencias de direcciones (CartoCiudad, del IGN) y el botón de tema.

Respecto al diseño de la Fase 1. Se mantienen las capas del diseño: datos en Parquet / GeoParquet sin servidor de base de datos, pipeline, modelo, una capa de acceso a datos que separa la aplicación del almacenamiento y la aplicación en Streamlit, todo en Python y en una imagen Docker con `data/` como volumen. Las diferencias son estas:

- La capa de acceso a datos (`data_access` en la Fase 1) es `src/backend/services.py`. Como estaba previsto para el MVP, no es una API ni un servicio aparte.
- La ingesta parte de los ficheros descargados en `data/raw` en lugar de descargarlos desde los scripts, porque varias fuentes piden cuenta (Copernicus Land) o se descargan desde visores y servicios WFS.
- La imagen Docker tiene cuatro puntos de entrada (`ingest`, `pipeline`, `train` y `app`) más uno para los tests, en lugar de tres.
- El mapa, el buscador y el botón de tema son componentes JavaScript propios dentro de Streamlit.
- El área (22 municipios en lugar de la ciudad de València), la celda de 100 m en lugar de 250 m y el modelo sin lluvia se explican en el apartado 2.

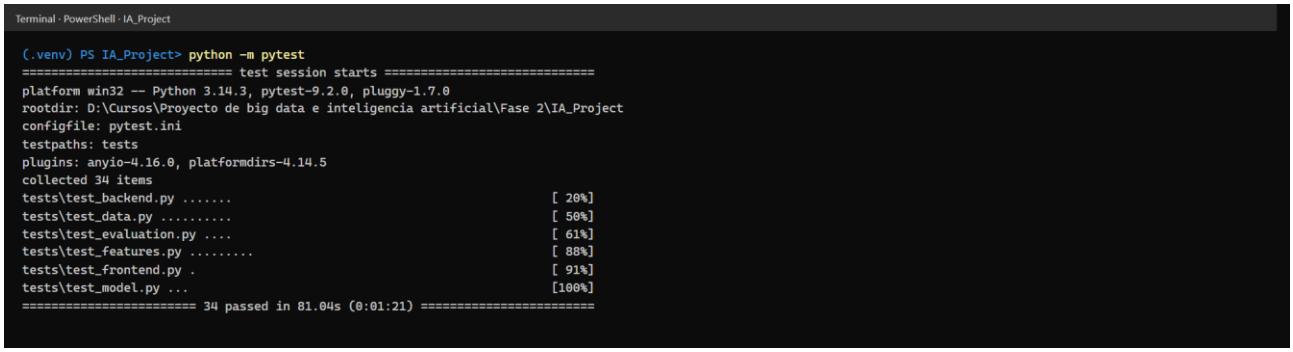
6. Índice de código

```
README.md
pytest.ini
.streamlit/config.toml
.env.example
.gitignore
.dockerignore
.github/workflows/ci.yml
src/
├── config.py
├── pipeline/
│   ├── ingest.py
│   ├── grid.py
│   ├── sources.py
│   ├── features.py
│   ├── blocks.py
│   ├── checks.py
│   ├── run.py
│   ├── evaluation.py
│   ├── train.py
│   └── predict.py
├── backend/
│   └── services.py
└── frontend/
    ├── app.py
    └── components/
        ├── cell_map.js / cell_map.css
        ├── place_search.js / place_search.css
        ├── theme_toggle.js / theme_toggle.css
        └── __init__.py
tests/
├── conftest.py
├── test_features.py
├── test_evaluation.py
├── test_data.py
├── test_model.py
├── test_backend.py
└── test_frontend.py
environment/
├── requirements.txt
```

```
├── Dockerfile
├── docker-compose.yml
└── entrypoint.sh
data/                          (no se versiona: raw está en Google Drive y el resto se genera)
docs/
├── DATOS.md
├── ENTREGA_NF3.md
├── MEJORAS_MODELO_NF3.md
├── modelo_datos.png
├── arquitectura.png
├── capturas/
└── entregas/
```

7. Repositorio versionado y reproducible

- **Enlace y versión:** https://github.com/Victorproyecto/IA_Project, tag `nf3-entrega` (rama `main`).
- **README:** requisitos, configuración y comandos.
- **Dependencias:** `environment/requirements.txt` con versiones fijadas. `.env.example` sin secretos.
- **Pruebas:** `python -m pytest` (34 tests, también en Windows, alrededor de 1 min) y CI en `.github/workflows/ci.yml`. Resultado:



- **Datos:** no se versiona nada de `data/`. Los originales (1,7 GB) están en Google Drive y el resto lo generan `ingest`, `run` y `train`. `docs/DATOS.md` indica la fuente, la licencia y cómo descargar cada fichero.

8. Explicación por módulo

Configuración (`src/config.py`)

- Rutas y nombres de los ficheros, áreas en metros (municipios del piloto por comarca), parámetros del pipeline y del modelo y criterios de éxito de la Fase 1, en un solo sitio.

Pipeline (`src/pipeline/`)

- `ingest.py`: une y recorta las descargas originales y genera `data/preparados/` (terreno, suelo sellado, PATRICOVA, barrios, etiqueta de la DANA con Copernicus EMS y la huella de la GVA reparada, municipios, cauces principales, calados de campo, suelo no urbano de Corine, geología del IGME y zona inundable del SNCZI). Con el MDT y los cauces calcula la altura sobre el cauce (HAND): sigue el agua de cada píxel cuesta abajo hasta su cauce, el mar o la Albufera, con `pysheds` para la dirección del flujo.
- `grid.py`: crea la cuadrícula de 100 m sobre el área piloto, fijada en metros, y calcula la pendiente.

- `sources.py`: lleva cada fuente a la celda (media de 25 puntos en rásteres, cruce espacial en vectores), asigna municipio y barrio por mayor superficie, calcula la distancia al cauce principal, la altura media sobre el cauce y la gravedad de lo inundado (completada con los calados de campo).
- `features.py`: invierte PATRICOVA para que más sea peor (solo para la comparación) y define la unidad territorial (barrio o distrito en València, municipio fuera).
- `blocks.py`: agrupa las celdas en bloques de 1 km y los reparte en 5 grupos de validación.
- `checks.py`: comprueba los datos antes de escribir.
- `run.py`: orquesta la preparación (área piloto y prueba externa) y escribe las tablas.
- `evaluation.py`: métricas, umbrales Bajo / Medio / Alto y comprobación con calados.
- `train.py`: validación cruzada de 2 modelos, selección, referencia trivial, comparación con PATRICOVA y el SNCZI, prueba externa en la Ribera, ficha y predicciones.
- `predict.py`: probabilidad y nivel por celda.

Backend (`src/backend/`)

- `services.py`: lectura de zonas y predicciones, cuadrícula compacta para el mapa (un byte por celda), límite del área por comarca, resumen por distrito o municipio y lista de lugares para las sugerencias del buscador.

Frontend (`src/frontend/`)

- `app.py`: página de Streamlit con el aviso de uso responsable y tres pestañas: el mapa con el buscador y la ficha de la celda al lado, la tabla de zonas con descarga en CSV y la ficha del modelo. Los números van en formato español.
- `components/`: el mapa de celdas (MapLibre GL JS, que dibuja cada celda como un cuadrado exacto, resuelve el clic en el navegador y tiene las capas de susceptibilidad, límite del área y lo inundado en la DANA), el buscador con sugerencias (municipios propios y calles de CartoCiudad) y el botón de tema claro / oscuro.
- `.streamlit/config.toml`: oculta las opciones de desarrollo de la barra de Streamlit (como el botón "Deploy").

Tests (`tests/`)

- `conftest.py`: ejecuta `run` y un `train` rápido una sola vez, sobre un recorte de 6 x 6 km y en un directorio temporal.
- `test_features.py` y `test_evaluation.py`: cuadrícula, pendiente, altura sobre el cauce, bloques, umbrales y métricas.
- `test_data.py`: comprobaciones de los Parquet generados y de la reparación de la huella de la GVA.
- `test_model.py`: contenido de la ficha del modelo.
- `test_backend.py` y `test_frontend.py`: funciones de `services.py` y arranque de la aplicación.

Entorno (`environment/`)

- `requirements.txt` con versiones fijadas, `Dockerfile`, `docker-compose.yml` y `entrypoint.sh` con los puntos de entrada `ingest`, `pipeline`, `train`, `app` y `test`.

9. Limitaciones actuales

- **Bugs:** no se han detectado errores en la aplicación ni en los tests.
- **Partes incompletas:** faltan las funciones de la Fase 1 que dependen de la lluvia (apartado 2).
- **Un solo episodio:** el modelo aprende de la DANA de 2024. Es un mapa de susceptibilidad aprendido de esa DANA, no una predicción de qué pasaría con otra lluvia.
- **No generaliza a otra cuenca:** en la Ribera su ROC-AUC es 0,59 y PATRICOVA y el SNCZI funcionan mucho mejor. La validación por bloques de 1 km mide si el modelo rellena bien los huecos dentro de una zona que conoce, y es algo optimista: con bloques de 3 km el F1 baja a 0,63.
- **F1 y proporción de inundadas:** el F1 de 0,72 hay que leerlo frente a 0,38, lo que daría marcarlo todo. Supera el objetivo de la Fase 1 (0,70) por poco, gracias a una variable que se añadió después de ver que sin ella no se llegaba, y esa mejora no se mantiene con bloques de 3 km.
- **Sin lluvia:** con un solo episodio, la lluvia no se puede aprender. El mapa es fijo, no es tiempo real.
- **Etiqueta:** Copernicus no ve el agua entre edificios; la huella de la GVA lo corrige en los cascos urbanos, pero por eso los calados de campo ya no son una validación independiente.
- **Comparación con los mapas oficiales favorable al modelo:** aprende de esta DANA; PATRICOVA y el SNCZI son anteriores y describen otra cosa (periodos de retorno).
- **Métricas posiblemente algo optimistas:** las variables de cauces (distancia y altura sobre el cauce) se diseñaron revisando este mismo episodio.
- **Atajos:** el backend no es un servicio aparte; los umbrales se fijan sobre las mismas predicciones de validación que se usan para medir.
- **Tema oscuro:** Streamlit no permite cambiar el tema desde Python; el botón guarda la elección igual que el menú de ajustes de Streamlit y recarga la página (se pierde la celda seleccionada). Si una versión futura cambiara esa clave, el tema se seguiría pudiendo cambiar desde el menú de ajustes.
- **Formato de números en la tabla de zonas:** sigue el idioma del navegador (coma decimal en español) para que las columnas sigan siendo ordenables; en el resto de la aplicación la coma decimal es fija.
- **Conexión:** el navegador necesita internet para el mapa de fondo, la librería del mapa y las sugerencias de direcciones.
- **Tests en la CI:** sin datos en el repositorio, la CI salta los tests que los necesitan.

10. Lista de comprobación de reproducibilidad

- Repositorio accesible para el profesorado
- Tag exacto indicado en la portada (`nf3-entrega`)
- README actualizado con requisitos y comandos
- Dependencias con versiones fijadas y `.env.example` sin secretos
- Tests automáticos (34) y CI
- Datos referenciados con fuente y licencia (docs/DATOS.md, todas comprobadas)
- Capturas de front, backend y flujo completo